

Predicting User Ratings of Video Games Using Amazon Reviews

Grace Boye-Williams, Will Pierce, Corey Rogers, Jude Shin

Data Selection

We selected data from Amazon User Reviews. Specifically, we decided to look at a subset of video game reviews. The McAuley lab collected this data from Amazon in 2023. The dataset is quite large with the video games subset containing 2.8M Users, 137.2k Items, and 4.6M Ratings. We thought this dataset would be interesting to look at because the data was well documented, there were a lot of metadata fields to work with, and there was a lot of potential for gaining insights on user behavior based on their reviews of products. After taking a closer look at the dataset we decided that we wanted to predict user ratings for items they had not yet reviewed based on how they have previously rated items.

Purpose Statement

User rating behavior is a form of user analytics that helps companies understand customer preferences and patterns. This data can influence business decisions, recommendation systems, and future product design choices, all of which impact user retention, engagement, and sales.

Data Preparation

Parsing the Data

The input format was one json element per line, so we parsed two text files: reviews, and metadata. There were helper functions that would parse different types of data (i.e. double, text, int, etc), and for each json item, would parse out the expected values. If anything went wrong in the parsing, null would be inserted. Any tuples that were null would then be filtered out to make sure we were working with a clean dataset.

The steps for both datasets are as follows:

- 1) Json text file input loaded from disk to RDD
- 2) `.map()` converting to a tuple
- 3) `.filter()` removing bad parsed tuples
- 4) `.map()` to key-value format (`parent_asin, (relevant data tuple)`)

Joining and Formatting the Data

To join the two datasets, both RDDs had to be converted to a key-value format, where the key was the parent_asin, and the value was the original tuple (for simplicity). All of the data was then joined on the parent_asin number that was present in both the reviews and metadata sets through the .join() function. Finally the data was formatted to the tuple of the form **((user_id, parent_asin), rating)**. This was to maintain consistency through both algorithms implemented. These tuples were then saved to disk so parsing and joining wouldn't have to be done every time we wanted to run the algorithms.

Algorithm Implementation

To predict user ratings we decided to implement 2 different algorithms to first find similar users to our target user and from there predict a rating for our target user. The algorithms selected were Cosine Similarity and Pearson's Correlation.

To implement Pearson's Correlation we wrote a function that takes in 2 User's ratings and returns a value between -1 and 1 indicating their similarity. 1 indicates perfectly similar, -1 is perfectly negative (opposite) similarity, and 0 indicates no correlation. To compute the correlation coefficient only reviews for items that both users rated are considered. Of the co-rated items each user's mean rating is computed. Then, for each shared item, we subtract the user's average from their individual item rating to see whether they rated that item above or below their usual rating. The numerator sums the products of these differences for both users. The denominator is the square root of the sum of the squared differences for the first user multiplied by the sum of the squared differences for the second user. The fraction is the Pearson's correlation coefficient.

To implement Cosine Similarity we designed a function that takes in two user ratings (converted to a vector form) and returns a value between -1 and 1 which indicates how similar the raw values of the two user vectors are. To compute, we only consider items both users rated, then we multiply each pair of user ratings together and sum those products to get the dot product. The denominator is the product of both vector lengths (magnitudes) of two rating vectors. The final fraction is the cosine similarity score. A value closer to one indicates users gave similar ratings, a value closer to zero indicates little to no alignment. Unlike Pearson's Correlation, Cosine Similarity does not subtract average rating, so it does not account for users that tend to rate higher or lower than others.

To use these formulas we implemented a function that takes in the data in the form of an RDD, a target user ID, the desired similarity function (Cosine or Pearsons), and an N value signifying the number of most similar users to return. The function collects all ratings from the target user and then runs the similarity function for the target users reviews and every other user that is not the target user in the RDD. The results are sorted by similarity score and the top N users with the highest similarity scores are returned.

Next, we can run a weighted average between those user reviews within the top N using the similarity score to the primary user and determine a predicted user review. Our driver configures and can run this process including the N parameter and type of similarity function used. To be able to measure our algorithm's accuracy, we have our driver choose and extract an existing line from the dataset, then

compare the actual review with the expected review from running the algorithm on it. Then, we have our driver do this multiple times and write to a file for analysis.

Results

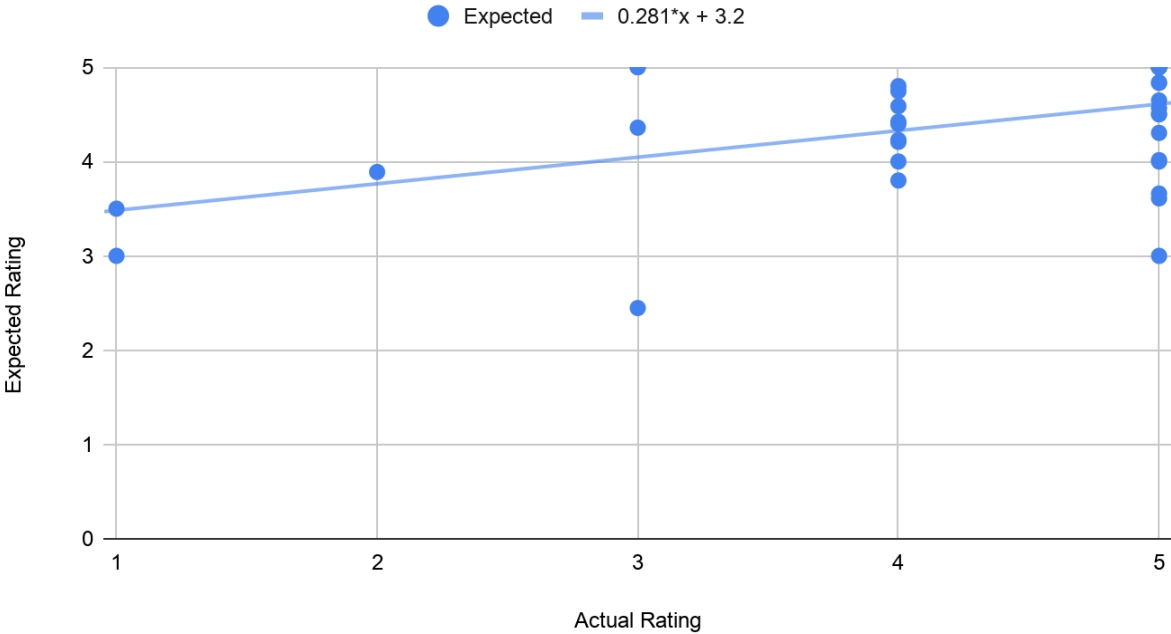
Over a run using the first 1000 random lines of the dataset. Only 39 of the runs were able to find neighbors to make a prediction.

Actual	Expected	Difference
4	4.745538907	-0.7455389069
5	5	0
5	5	0
5	5	0
5	4.836522121	0.1634778789
5	5	0
5	4.646980921	0.3530190786
5	5	0
5	5	0
5	5	0
4	4.798720549	-0.7987205487
5	4.833333333	0.1666666667
5	4	1
5	5	0
5	4.5	0.5
5	5	0
5	3	2
3	5	-2
5	5	0
5	5	0
4	4.42116461	-0.4211646096
2	3.889201201	-1.889201201
1	3	-2
1	3.5	-2.5
4	4.398854288	-0.3988542879

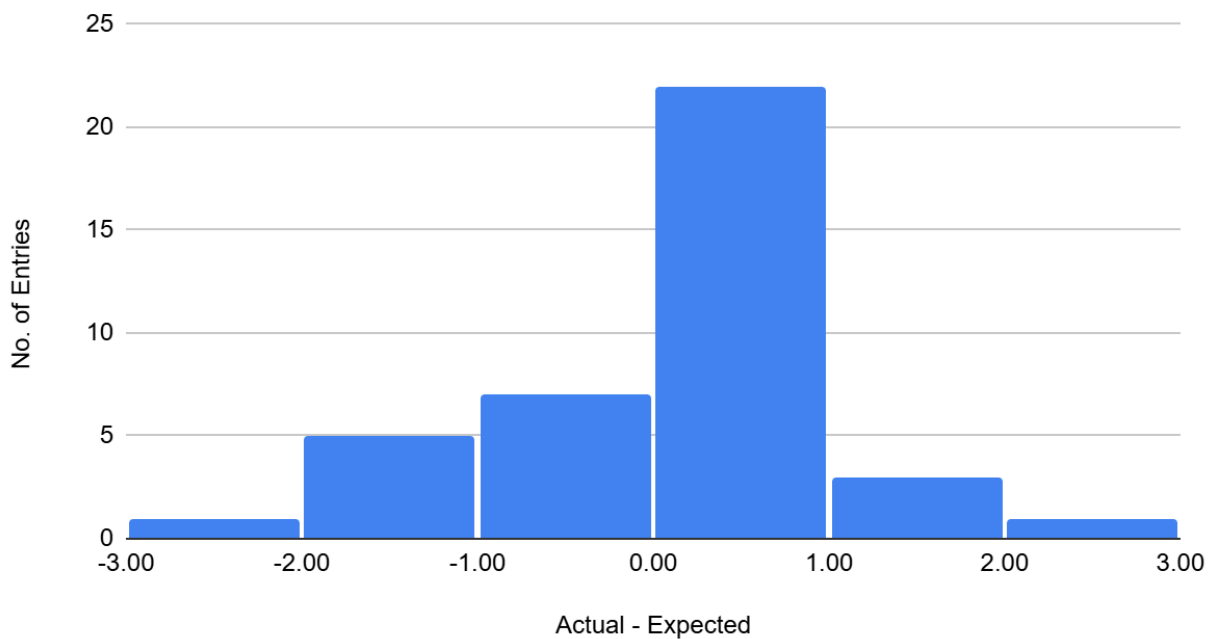
3	5	-2
4	4.208435772	-0.2084357719
3	2.447264814	0.552735186
5	4.56508419	0.4349158099
5	3.660188621	1.339811379
4	4.227568596	-0.2275685959
4	4.586508854	-0.5865088538
5	4.303065791	0.696934209
3	4.358888394	-1.358888394
5	4.504782802	0.4952171979
4	3.798839717	0.2011602828
5	4.014297733	0.985702267
4	4	0
5	3.609728479	1.390271521

We plotted these results in a scatterplot and histogram:

Actual vs. Expected Ratings



Differences Between Actual and Expected



We can count the ranges of expected values produced by our program:

Expected Range	Count
2.0–2.99	1
3.0–3.99	7
4.0–4.49	10
4.5–4.99	13
5	8

We can also average the differences. Despite the median being Actual higher than Expected, the mean difference between Actual and Expected was -0.124, indicating on average Expected would overestimate possible user reviews.

From our findings, we can conclude that our algorithm's process of comparing similar users is able to predict the actual user reviews, if to a degree. The margin of error was very low on average, and our scatterplot of the data does not point towards a random distribution.

One other noteworthy observation is how the reviews that were able to be calculated in this data set were overwhelmingly positive, on average. Because being able to calculate a predicted review generally requires a user to leave multiple reviews on products, it could indicate that the overwhelming majority of reviews for Amazon game products are positive and/or many users who leave negative reviews on

products often do not review other products that others do. Thus, only 4% of user reviews have a user and product who both leaves reviews on other products and gets reviews from many other users.

To be able to make use of more of our data and improve our prediction model, we could perform our comparison-building to greater depth, and make inferences on users in a network even if they haven't rated the exact same products.

Challenges Faced

Determining how to join data was a big challenge. Both cartesian products were used, as well as the `.join()` function, and the `join` function was significantly faster. The case statement to format the join was significantly slower if the case statement tuples were very large (i.e. `((_, _, _, _, _, rating, parent_asin, _), (_, _, _, _))`) could be simplified to `(tuple_1, tuple_2)`). This change significantly improved the performance of the parsing and joining step.

Figuring out interesting/useful metrics was also an area that we spent some time trying to figure out. The `.join()` allows us to include other information like the price of the product that was being reviewed, or the average rating at the time the user reviewed a product. We had some trouble choosing what information we would want to include in this recommendation algorithm. These algorithms could be extrapolated to handle other pieces of information in the dataset like the pieces mentioned above, however, we ended up deciding that the clearest way to go was to just use the numerical rating between 0.0 and 5.0.